

INTELIGENCIA ARTIFICIAL I

PEC1 – 2009_2 Prueba de Evaluación Continua

- Para dudas y aclaraciones sobre el enunciado, debéis dirigiros al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero Word, OpenOffice, PDF o RTF utilizando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntad el fichero a un mensaje dirigido en el buzón **Entrega de actividades**.
- El nombre del fichero tiene que ser *ApellidosNombre_IA1_PEC1* con la extensión .doc (Word), .sxw (OpenOffice), .pdf (PDF) o .rtf (RTF), según el formato en que hagáis la entrega.
- La fecha límite de entrega es el: **13 de Octubre** (a las 24 horas).
- **Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.**

Enunciado:

Un acertijo consiste en a partir de cuatro números y un resultado determinar las operaciones de suma o resta que hay que hacer con éstos para encontrar el resultado. Por ejemplo:

Números: 1,4,3,2

Resultado: 0

Operaciones: 4-3-2+1

Suponiendo que queremos resolver el acertijo como un problema de búsqueda responde a las siguientes preguntas:

- 1.- Propón un espacio de estados para resolver el problema del ejemplo.
- 2.- Según la propuesta anterior, ¿cuántos estados se consideraran?
- 3.- Se trata de una búsqueda a ciegas de un estado objetivo. Aplicando el algoritmo de búsqueda en amplitud encuentra la solución al acertijo.
 - 3.1.- ¿Cuántos operadores tendrás? ¿Cuáles serán estos operadores? ¿Cómo relacionan los operadores los estados descritos anteriormente?
 - 3.2.- Da la definición del estado inicial y la del estado objetivo.
 - 3.3.- Realiza una representación del árbol de búsqueda.
 - 3.3.a - ¿A qué nivel de profundidad has encontrado la solución?
 - 3.3.b - ¿Cuántos nodos has generado?
 - 3.3.c – Muestra la evolución de la cola de pendientes y di la cantidad de memoria necesaria (en número de nodos). Considera que se evalúa si un nodo es estado final cuando este se introduce en la lista de pendientes y no cuando se coge para expandir tal y como se indica en los módulos.
 - 3.3.d - Nodos pendientes de expansión al encontrar la solución.

4.- Ahora aplicando el algoritmo de búsqueda en profundidad encuentra la solución al acertijo y analiza la eficiencia de este algoritmo comparándola con la del anterior.

SOLUCIÓN:

1.- *Propón un espacio de estados para resolver el problema del ejemplo.*

Un estado vendrá representado por una tupla (s_1 ; s_2 ; s_3 ; s_4) en la que $s_i \in [+1, -1, 0]$ representa el signo del número en la posición i . El valor 0 indica que aún no se ha asignado ningún signo.

2.- *Según la propuesta anterior, ¿cuántos estados se consideraran?*

El tamaño del espacio de estados es 3^n , donde n es la cantidad de números que nos da el problema, en este caso: $3^4 = 81$. Ahora bien, el número de nodos a generar en el algoritmo en amplitud depende de los operadores que se definan. Una posible opción sería que a cada nivel se asigne un signo al número i -ésimo. En el ejemplo, en el primer nivel asignaremos signo al 1, en el segundo al 4, en el tercer al 3 y por último al dos. Así pues, tenemos un factor de ramificación de 2, con una profundidad de 5, con la cual cosa obtenemos un número máximo de nodos de $2^5 = 32$. Este valor es muy inferior a los 81, ya que hay estados a los que con esta representación de los operadores nunca se generarán.

3.- *Se trata de una búsqueda a ciegas de un estado objetivo. Aplicando el algoritmo de búsqueda en amplitud encuentra la solución al acertijo.*

3.1.- *¿Cuántos operadores tendrás? ¿Cuáles serán estos operadores? ¿Cómo relacionan los operadores los estados descritos anteriormente?*

Tendré 8 operadores:

Operador 1: +(1000)
Operador 2: - (1000)
Operador 3: +(0100)
Operador 4: - (0100)
Operador 5: +(0010)
Operador 6: - (0010)
Operador 7: +(0001)
Operador 8: - (0001)

Cada uno de los operadores suma a la tupla de partida la tupla especificada en cada operador.

Para evitar bucles, un operador solo se podrá aplicar si previamente se ha aplicado uno de índice $i-1$ o $i-2$ y una vez aplicado no se podrá volver a aplicar. Dicho de otra manera al nivel i se podrán aplicar los operadores $i+1$ o $i+2$ únicamente.

3.2.- *Da la definición del estado inicial y la del estado objetivo.*

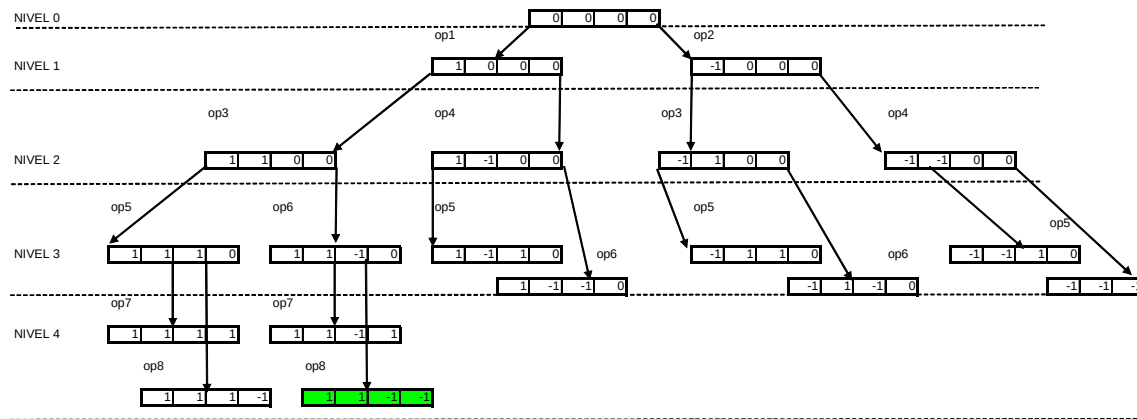
El estado inicial de la tupla es (0,0,0,0).

El estado final viene representado implícitamente, de forma que se debe cumplir:

$s_i \neq 0$
 i

$$(1 \ 4 \ 3 \ 2)^* \begin{pmatrix} s1 \\ s2 \\ s3 \\ s4 \end{pmatrix} = 0$$

3.3.- Realiza una representación del árbol de búsqueda.



3.3.a - ¿A qué nivel de profundidad has encontrado la solución?

3.3.b - ¿Cuántos nodos has generado?

3.3.c – Muestra la evolución de la cola de pendientes y di la cantidad de memoria necesaria (en número de nodos). Considera que se evalúa si un nodo es estado final cuando este se introduce en la lista de pendientes y no cuando se coge para expandir tal y como se indica en los módulos.

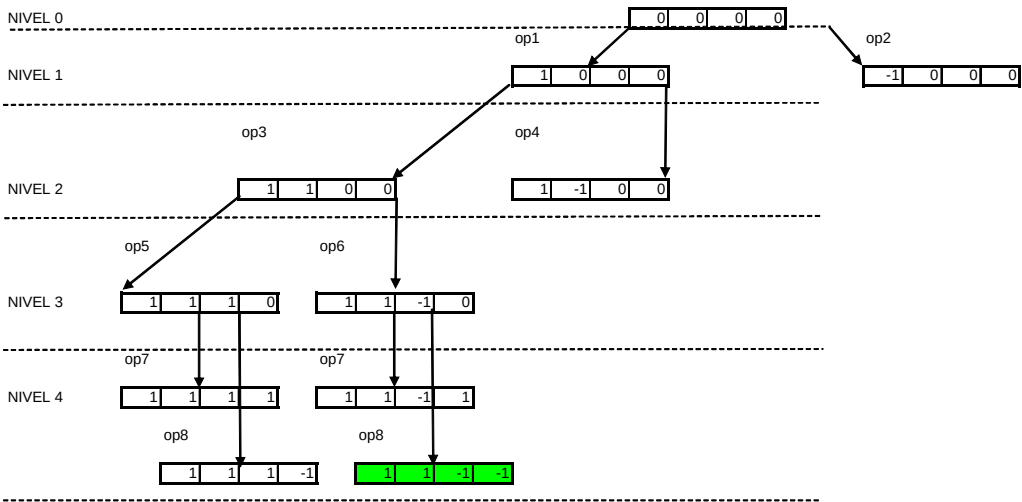
Cola pendientes
(0000)
(1000) (-1000)
(-1000) (1100) (1-100)
(1100) (1-100) (-1100) (-1-100)
(1-100) (-1100) (-1-100) (1110) (11-10)
(-1100) (-1-100) (1110) (11-10) (1-110) (1-1-10)
(-1-100) (1110) (11-10) (1-110) (1-1-10) (-1110) (-11-10)
(1110) (11-10) (1-110) (1-1-10) (-1110) (-11-10) (-1-110) (-1-1-10)
(11-10) (1-110) (1-1-10) (-1110) (-11-10) (-1-110) (-1-1-10) (1111) (111-1)
(1-110) (1-1-10) (-1110) (-11-10) (-1-110) (-1-1-10) (1111) (111-1) (11-11) (11-1-1)

La memoria necesaria es de 10 nodos, ya que se ha realizado la simplificación de evaluar al añadir a la cola de pendientes.

3.3.d - Nodos pendientes de expansión al encontrar la solución.

10 nodos

4.- Ahora aplicando el algoritmo de búsqueda en profundidad encuentra la solución al acertijo y analiza la eficiencia de este algoritmo comparándola con la del anterior.



Cola pendientes
(0000)
(1000) (-1000)
(1100) (1-100) (-1000)
(1110) (11-10) (1-100) (-1000)
(1111) (111-1) (11-10) (1-100) (-1000)
(111-1) (11-10) (1-100) (-1000)
(11-10) (1-100) (-1000)
(11-11) (11-1-1) (1-100) (-1000)

El algoritmo es más eficiente ya que todas las soluciones tienen el mismo nivel de profundidad y aplicando el algoritmo en profundidad se necesita menos memoria, el número máximo de nodos a almacenar es de 5 comparado con los 10 de antes.